Track 2: Quantum Machine Learning

Variational classifiers, trainability and fair benchmarking

Qiskit Fall Fest Hackathon · Oct 9, 2026 · IISER Bhopal (EECS Club and Physics Club)

Read this first. You have one week, a laptop, and AI assistants. The ten problem statements are suggestions, not a menu you must order from. You may pick one, extend one, combine several, or invent something more ambitious. Original, advanced and cross-track projects are welcome and will be judged on the same criteria.

What we reward. A project that works, that you can explain line by line, and that is honestly compared against a classical alternative. Using AI to write code is allowed and encouraged; being unable to explain or verify that code is not.

How to read the problem statements

Every problem has the same parts: a short background, the problem itself, the core tasks (what a complete submission does), stretch goals (what makes it excellent), and a hint on tools. Treat the core tasks as the minimum, not the ceiling.

Simulators first, hardware as a bonus. Everything here can be done on a simulator (Qiskit Aer or the statevector tools). Running a small version on real IBM hardware, with a clear comparison against the simulator, earns extra credit in any track.

This track contains: **Problem 3, Problem 4**.

Problem 3: Build and benchmark a variational quantum classifier

Background

A variational quantum classifier encodes data into a quantum circuit, applies trainable gates, measures, and updates the gate angles to reduce a loss. Whether this ever beats classical models is an open research question, so the interesting work is a fair, careful test.

Problem

Build a VQC from scratch for a classification task of your choice, and find out when it helps, when it fails, and why.

Core tasks (the minimum for a complete submission)

1. Pick a dataset (for example a reduced version of Iris, a two-class slice of a standard dataset, or a synthetic one that is not linearly separable). Explain how you reduced it to fit a few qubits.
2. Implement the three parts yourself: a feature map, a trainable ansatz and a readout. Train with a classical optimiser.
3. Report training and test accuracy and show a learning curve. Use a held-out test set.
4. Compare against at least two classical baselines with a similar number of parameters (for example logistic regression and a small neural network).

Stretch goals (what makes it excellent)

- Study the choice of feature map and ansatz depth, and show which design choices matter most.
- Compare the VQC with a quantum kernel method.
- Test robustness to shot noise and to a hardware noise model.
- Visualise the decision boundary in two dimensions.

Hints on tools

Qiskit Machine Learning has ready-made VQC and kernel classes, but you are expected to open the box: show the circuit and explain each gate. An entry that reports "no advantage, and here is why" is valued over one that hides weak results.

Problem 4: A quantum neural network that avoids barren plateaus

Background

Deep, randomly initialised quantum circuits suffer from barren plateaus: the gradients shrink exponentially with the number of qubits, so training stalls. This is one of the main obstacles to scaling quantum neural networks. Several mitigation strategies exist, including starting from a near-identity circuit, training layer by layer, using local rather than global cost functions, and structured ansatz designs.

Problem

Show a barren plateau, then build and evaluate a quantum neural network that mitigates it.

Core tasks (the minimum for a complete submission)

1. Measure how the variance of a gradient component scales with the number of qubits (for example 2 to 10) and the circuit depth, for a randomly initialised hardware-efficient ansatz. Plot it on a log scale.
2. Implement at least one mitigation strategy and repeat the measurement, showing that the gradient variance decays more slowly.
3. Train your mitigated network on a small task (classification or regression) and compare the final loss and convergence speed with the unmitigated version.
4. Explain, in words and with a plot, why your strategy works.

Stretch goals (what makes it excellent)

- Compare two or three strategies on the same task (for example identity-block initialisation, layer-wise training and a local cost).
- Show that a global cost function causes a plateau even for a shallow circuit.
- Investigate whether noise creates a plateau of its own, and what that means for hardware.
- Combine with another track, such as a quantum neural network that learns to decode or correct errors (see the combined-project ideas).

Hints on tools

Use the parameter-shift rule or automatic differentiation on a simulator to get gradients. Average over many random initialisations; a single run tells you nothing about a variance.

Go beyond: combine tracks or invent your own

The most impressive entries will not stop at one box. None of these is required, and you are free to propose something entirely different.

Ideas that involve Track 2

Idea	Tracks combined	What makes it interesting
A neural-network decoder	QNN and error correction	Train a quantum or classical network to decode Steane syndromes, and compare with the lookup table.
Barren-plateau-aware chemistry	VQE and QNN mitigation	Apply identity-block initialisation or layer-wise training to a larger VQE ansatz.

Open invitation. If you have your own idea, such as a quantum game, an algorithm tutor, a benchmark of a quantum algorithm from our workshop (Grover, phase estimation, the QFT), or a tool that makes any of the above easier for the next student, propose it. Write three sentences on the problem, the method, and how you will know it worked, and check with the organisers early in the week.

How to scope a combined project. Get the smallest version of each part working separately first, then connect them. A small, honest, working combination beats an ambitious one that does not run.

What every team submits

Whichever problem you choose (or invent), your submission has the same five parts:

- A code repository or notebook that runs from a clean start, with versions of the packages listed.
- A write-up PDF (two to four pages) covering the problem, method, results and an honest limits section.
- At least one figure that shows the main result.
- A short AI-use note: which tools you used, one thing the AI got wrong, and how you caught it.
- A five-minute demo followed by questions; every team member must be able to take questions.

Bonus. Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator.

Full submission rules are in *docs/guidelines_and_submission_rules.pdf*.

How you are judged

The same scoring applies to every problem and to original projects alike.

Criterion	Points	What judges look for
Correctness and verification	30	Results checked against an exact or classical reference, tests, sensible error bars
Understanding and insight	25	Clear explanation of why the method works or fails, shown in the write-up and in the live questions
Honest comparison and limits	20	A fair classical baseline, stated assumptions, no unsupported claims of quantum advantage
Ambition and originality	15	Stretch goals, combined tracks, or a new idea beyond the suggested statements
Clarity of demo and code	10	Readable, reproducible code and a clear five-minute story
Total	**100**	Bonus points are available on top for a real IBM hardware run (see Bonus).

On using AI. AI assistants are welcome at every step. The AI-use note and the live questions exist because the person submitting is responsible for the result: you should be able to explain each circuit, formula and plot, and show how you verified it.